

Tarea: referencias, direcciones de memoria y punteros

Programación I

Nombre del estudiante: _____
Matrícula: _____
Grupo: _____
Fecha de entrega: _____

Objetivo

Practicar el uso de referencias, direcciones de memoria y punteros en C++, comprendiendo la relación entre una variable, su dirección de memoria y el acceso indirecto a su contenido.

En esta tarea solamente se utilizarán variables normales, arreglos de tamaño fijo, referencias y punteros. Todavía no se utilizará memoria dinámica.

Parte 1. Ejercicios de W3Schools

Ingresa al siguiente sitio:

https://www.w3schools.com/cpp/cpp_exercises.asp

Realice **todos los ejercicios** de las siguientes categorías:

1. **References**
2. **Memory Address**
3. **Pointers**
4. **Dereference**
5. **Modify Pointers**

Importante

No se deben realizar las categorías **Memory Management** ni **New and Delete**, ya que la memoria dinámica todavía no se ha estudiado en clase.

Puede realizar varios intentos en cada ejercicio. Lo importante es completar las actividades y comprender por qué funciona cada respuesta.

Parte 2. Ejercicios de programación

Además de los ejercicios de W3Schools, deberá desarrollar los siguientes dos programas en C++.

Ejercicio 1. Estadísticas de un arreglo mediante punteros

Desarrolle un programa que almacene diez números enteros en un arreglo y calcule:

- La suma de los elementos.
- El promedio.
- El valor mayor.
- El valor menor.

El programa deberá utilizar una función con la siguiente estructura:

```
1 void calcularEstadisticas(  
2     const int* arreglo,  
3     int cantidad,  
4     int* suma,  
5     double* promedio,  
6     int* mayor,  
7     int* menor  
8 );
```

La función deberá entregar los resultados al programa principal mediante los punteros recibidos.

Requisitos

- a) El arreglo debe tener un tamaño fijo de diez elementos.
- b) Los números deben ser solicitados al usuario.
- c) Dentro de la función, los elementos deben recorrerse mediante aritmética de punteros.
- d) Dentro de la función no se permite utilizar la notación `arreglo[i]`.
- e) La función debe ser de tipo `void`.
- f) Los resultados deben imprimirse desde la función `main`.
- g) No se permite utilizar variables globales.

Ejemplo de ejecución

```
1 Ingrese 10 numeros:
2 Numero 1: 8
3 Numero 2: 3
4 Numero 3: 12
5 Numero 4: 5
6 Numero 5: 9
7 Numero 6: 2
8 Numero 7: 15
9 Numero 8: 6
10 Numero 9: 4
11 Numero 10: 10
12
13 Suma: 74
14 Promedio: 7.4
15 Mayor: 15
16 Menor: 2
```

Ejercicio 2. Ordenamiento de un arreglo mediante punteros

Desarrolle un programa que solicite ocho números enteros y los almacene en un arreglo. Posteriormente, el programa deberá ordenar los elementos de menor a mayor utilizando punteros.

El programa deberá incluir, como mínimo, las siguientes funciones:

```
1 void intercambiar(int* numero1, int* numero2);
2
3 void ordenarArreglo(int* inicio, int cantidad);
4
5 void imprimirArreglo(const int* inicio, int cantidad);
```

Requisitos

- El arreglo debe tener un tamaño fijo de ocho elementos.
- La función `intercambiar` debe modificar directamente los dos valores recibidos.
- La función `ordenarArreglo` debe ordenar los números de menor a mayor.
- El recorrido del arreglo debe realizarse mediante punteros.
- Dentro de las funciones no se permite utilizar la notación `arreglo[i]`.
- Se puede utilizar el método de ordenamiento burbuja o selección.
- Se debe imprimir el arreglo antes y después de ordenarlo.
- No se permite utilizar `std::sort`.

- i) No se permite utilizar memoria dinámica.
- j) No se permite utilizar variables globales.

Ejemplo de ejecución

```
1  Ingrese 8 numeros:
2  Numero 1: 17
3  Numero 2: 4
4  Numero 3: 25
5  Numero 4: 1
6  Numero 5: 9
7  Numero 6: 13
8  Numero 7: 6
9  Numero 8: 2
10
11 Arreglo original:
12 17 4 25 1 9 13 6 2
13
14 Arreglo ordenado:
15 1 2 4 6 9 13 17 25
```

Restricciones generales

En los dos ejercicios de programación está prohibido utilizar:

- El operador `new`.
- El operador `delete`.
- Las funciones `malloc`, `calloc`, `realloc` o `free`.
- Arreglos creados dinámicamente.
- Contenedores de la biblioteca estándar, como `vector`.
- Variables globales.
- Funciones recursivas.

Se deberán utilizar únicamente arreglos de tamaño fijo declarados dentro de la función `main`.

Evidencias que se deben entregar

El estudiante deberá entregar un único archivo en formato **PDF** que contenga:

1. Portada con nombre completo, matrícula, grupo y nombre de la tarea.
2. Capturas de pantalla que demuestren que se completaron las categorías solicitadas de W3Schools.
3. Código fuente completo del ejercicio 1.
4. Captura de la compilación y ejecución del ejercicio 1.
5. Código fuente completo del ejercicio 2.
6. Captura de la compilación y ejecución del ejercicio 2.
7. Una conclusión personal de al menos cinco líneas sobre:
 - Qué es un puntero.
 - Qué representa una dirección de memoria.
 - Para qué sirve desreferenciar un puntero.
 - Qué dificultad encontró durante la actividad.

Además del PDF, se deberán entregar los siguientes archivos:

- ejercicio1.cpp
- ejercicio2.cpp

Criterios de evaluación

Criterio	Valor
Evidencia de los ejercicios de W3Schools	30 %
Funcionamiento y cumplimiento de requisitos del ejercicio 1	25 %
Funcionamiento y cumplimiento de requisitos del ejercicio 2	30 %
Orden, claridad y comentarios en el código	5 %
Evidencias de ejecución y conclusión personal	10 %
Total	100 %

Consideraciones finales

- Los programas deben compilar y ejecutarse correctamente.
- El código deberá estar correctamente indentado.
- Las variables y funciones deberán tener nombres descriptivos.
- Se pueden realizar varios intentos en los ejercicios de W3Schools.
- Las capturas deben ser claras y mostrar el resultado completo.
- El estudiante deberá ser capaz de explicar el funcionamiento de su código.
- Los trabajos copiados total o parcialmente serán evaluados de acuerdo con las políticas establecidas en el curso.

El objetivo no es solamente obtener el resultado correcto, sino comprender cómo los punteros permiten acceder y modificar información mediante direcciones de memoria.